Jone Gorman AI Learning Guide: Start from Scratch, Learn by Building

2026-10-19

Contents

1. Understand computers **2**

2. Learn your operating system **2**

3. Learn Python **2**

4. Manage development environments **2**

5. Move from machine learning to deep learning **3**

6. Understand large language models **3**

7. Reinforcement learning: foundations and PPO **4**

8. Explore agents and applications **5**

9. Build a sustainable learning practice **5**

Expensive courses, unfamiliar terminology, and a constant stream of new technology can make getting started with AI feel overwhelming. This guide begins from the perspective of a learner with a limited budget and little computing experience, breaking "I want to learn AI" into manageable steps.

From understanding a computer to writing a first Python script, reproducing a model, and finishing a small project, each step can leave you with code and notes of your own. Courses provide an entry point; hands-on work connects the ideas.

1. Understand computers

Start with the basic parts of a computer. Distinguish the CPU, memory, storage, and GPU, and develop a rough understanding of how programs, files, and hardware work together. You do not need high-performance hardware or a complete computer science education to begin.

- Introduction to computers, in Chinese

Practical checkpoint: Find your device's processor, memory, and disk capacity, and locate frequently used files. Memory and disk storage are different; installing software primarily consumes storage space.

2. Learn your operating system

Begin with familiar Windows operations, then explore the operating system, terminal, directories, and permissions. Learn Linux when a course or project calls for it. Dual boot is one route included in the original document; a virtual machine or WSL can also provide a place to practice.

- Dual-boot installation tutorial, in Chinese
- Basic Linux operations, in Chinese

Practical checkpoint: Use a terminal to display and change directories, create a folder, and run a program. Understand what each command does.

3. Learn Python

Building a foundation in one language is an accessible starting point. This route uses Python: learn variables, conditionals, loops, functions, common data structures, imports, and basic debugging before moving into machine-learning code.

- Setting up Python and VS Code, in Chinese
- Python fundamentals, in Chinese

Practical checkpoint: Write a small script that reads a file, processes its contents, and saves a result. After following a tutorial, change the input or add a feature and explain the program's behavior.

4. Manage development environments

Projects often require different versions of Python and their dependencies. Miniconda provides a relatively small initial installation for creating project-specific environments; conda or pip within the environment can install the packages you need. The actual storage difference from Anaconda depends on what is installed, rather than a universal size ratio.

- Miniconda tutorial, in Chinese
- Official conda environment-management documentation

The original document uses Python 3.10 for its first environment. This is an example of the commands, not a version requirement for every project; follow the course or dependency file you are

working with.

```
conda create -n aienv python=3.10
conda activate aienv
python --version
python -m pip --version
```

Practical checkpoint: Confirm that your terminal, editor, and package installation use the same environment. Record the project's Python version and dependency configuration.

5. Move from machine learning to deep learning

Mathematics, code, and experiments come together in AI. If a derivation is initially difficult, run a small example and then connect the equations with the inputs, outputs, and implementation. Courses move at different speeds; combine resources as needed without turning completion speed into the goal.

Courses and practical material

- Mu Li: Dive into Deep Learning, in Chinese
- Abhishek Thakur: Approaching (Almost) Any Machine Learning Problem, author's repository
- AAAMLP Chinese translation
- RethinkFun deep-learning tutorials, in Chinese

Git, GitHub, and hardware

Learn version control alongside programming: Git records changes, while GitHub can host and share repositories. Start with examples suited to your existing hardware, and evaluate GPU needs when a project requires it. The PC-building video from the original document is retained as supplementary hardware material.

- Introduction to GitHub, in Chinese
- Introduction to Git, in Chinese
- PC-building tutorial by Hardware Tea Talk, in Chinese

Practical checkpoint: Finish a modeling experiment with a data split, evaluation metric, and recorded results, then commit your implementation to a Git repository.

6. Understand large language models

Start with the Transformer and connect attention, tensor shapes, and the forward pass. Implementing a teaching model line by line before studying architectures such as Llama makes the modules in a paper more concrete. The sequence here is a learning path, rather than a ranking of technologies by recency or performance.

Transformer and Llama

- Attention Is All You Need walkthrough
- Implementing a Transformer in PyTorch
- Llama architecture walkthrough
- Implementing Llama in PyTorch

Model architecture

- RoPE: rotary position embeddings
- MoE: mixture of experts
- RMSNorm

Fine-tuning, inference, and efficiency

LoRA is a parameter-efficient fine-tuning method that can be used for supervised fine-tuning. KV caching, FlashAttention, and PagedAttention address different aspects of attention computation, inference, or memory management. Distinguish the problem each technique tackles.

- LoRA fine-tuning
- KV caching
- FlashAttention
- PagedAttention

Practical checkpoint: Implement and explain an attention module at a teaching scale, recording key tensor shapes. For each subsequent technique, identify which part of the system it changes and what conditions it introduces.

7. Reinforcement learning: foundations and PPO

Understand states, actions, rewards, policies, and value functions before moving into algorithms and code. PPO is a focus of this route. Its performance depends on the task, implementation, and evaluation setup; it is not universally the best algorithm for every problem.

- Shusen Wang: Deep Reinforcement Learning, in Chinese
- PPO tutorial, in Chinese
- The 37 Implementation Details of Proximal Policy Optimization

Learn implementation details alongside experiment tracking: inspect learning curves and record random seeds, hyperparameters, and the runtime environment. TensorBoard and Weights & Biases (W&B), mentioned in the original document, can support visualization. Begin with records you can save and revisit.

The original document also includes a community post about connecting to W&B. The post's entry URL is retained without personal sharing parameters. Its content and availability depend on the platform.

Distinguish inference engines from RL frameworks

- vLLM documentation: primarily for LLM inference and serving; it can also provide generation within an RL system.
- verl repository and official documentation: a framework for reinforcement learning with large language models.

Practical checkpoint: Complete a small PPO experiment with saved configuration and learning curves, then identify the separate roles of training, generation, and evaluation in an LLM reinforcement-learning system.

8. Explore agents and applications

Once you can read Python, manage dependencies, and understand basic model calls, explore how agents connect tools and external environments. This is the application stage of the original route; a small, clearly bounded project can bring the earlier material together.

- Hugging Face AI Agents Course: introduction
- Course livestream page included in the original document

Practical checkpoint: Choose a task whose result you can check. Implement a tool call, record the input, tool response, and final output, then verify that the model actually used the returned information.

9. Build a sustainable learning practice

English reading and listening help you access courses, papers, and official documentation directly. When a translation is unclear, return to the original source, code, and actual execution results.

Creators mentioned in the original document include:

- **Umar Jamil**: algorithm implementations and paper walkthroughs.
- **Andrej Karpathy**: hands-on neural-network and language-model code.
- **Mu Li**: deep-learning courses, paper reading, and engineering practice.

Search around problems that interest you, read documentation, reproduce code, and learn to describe your questions clearly when asking for help. Preserve environments, changes, and results so that your work remains something you can revisit and build on.

LLMs are an important part of this path, and AI extends beyond them. As the focus of the field changes, the ability to find information, understand a problem, and implement an idea will continue to support the next thing you learn.

Start with curiosity, stay focused, and learn alongside people who share your interests. **Get your hands dirty.**